Programación Avanzada
IIC2233 2026-2

Anuncios

5 de noviembre de 2026



- Recuerden **marcar su salida con la TUC**, sino quedarán como ausentes.

Control de Entrada

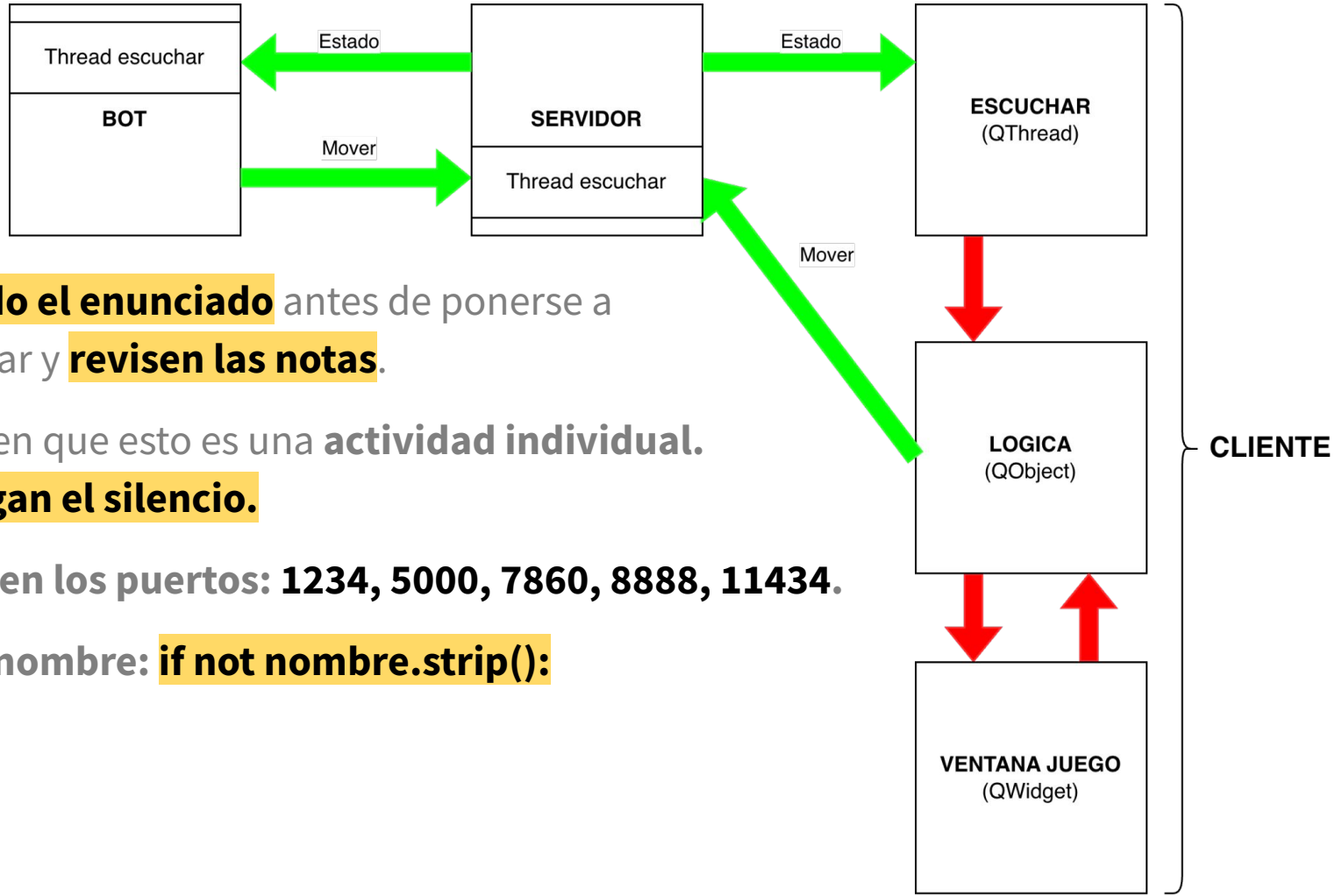
Estudiantes de la **sección 2**, recuerden **indicar su sección** junto a su nombre y RUT.



`git add, git commit, git push`

Comentarios AC 8

- **Lean todo el enunciado** antes de ponerse a programar y **revisen las notas**.
- Recuerden que esto es una **actividad individual. Mantengan el silencio.**
- **No utilicen los puertos: 1234, 5000, 7860, 8888, 11434.**
- **Revisar si un nombre es puros espacios: if not nombre.strip():**



- **Lean todo el enunciado** antes de ponerse a programar y **revisen las notas**.
- Recuerden que esto es una **actividad individual.** **Mantengan el silencio.**
- **No utilicen los puertos: 1234, 5000, 7860, 8888, 11434.**
- **Revisar nombre: if not nombre.strip():**

Cierre

Actividad 8: Interfaces Gráficas 2



Responsabilidades *Front-end*, *Back-end* y Servidor

Front-end
Las dos ventanas

Recibir del usuario:
- Nombre
- Teclas W y S (P en el bonus)

Mostrar el estado:
- Pelota y paletas
- Marcador y reloj
- Avisos y mensaje final

Back-end
Lógica

Comunicarse con el servidor:
- Escuchar en un QThread
- Enviar comandos

Manejar el estado local:
- Teclas y posición de la paleta
- Reloj y timers

Servidor
Ya venía implementado

Fuente de verdad:
- Física de la pelota
- Goles y marcador
- Fin de la partida

Avisa a ambos clientes:
- estado, gol, fin
- rival_desconectado

¿Por qué escuchar en un QThread?

EscucharQThread

```
class EscucharQThread(QThread):
    senal_mensaje = pyqtSignal(dict)
    senal_desconectado = pyqtSignal()

    def run(self):
        try:
            while True:
                mensaje = recibir(self.socket_cliente)
                self.senal_mensaje.emit(
                    json.loads(mensaje.decode('utf-8'))
                )
        except (ConnectionError, OSError):
            self.senal_desconectado.emit()
```

¿Por qué escuchar en un QThread?

¿Por qué no llamar a `recibir` en el thread principal?

recv se bloquea hasta que llega un mensaje: la ventana se congela y deja de responder a las teclas.

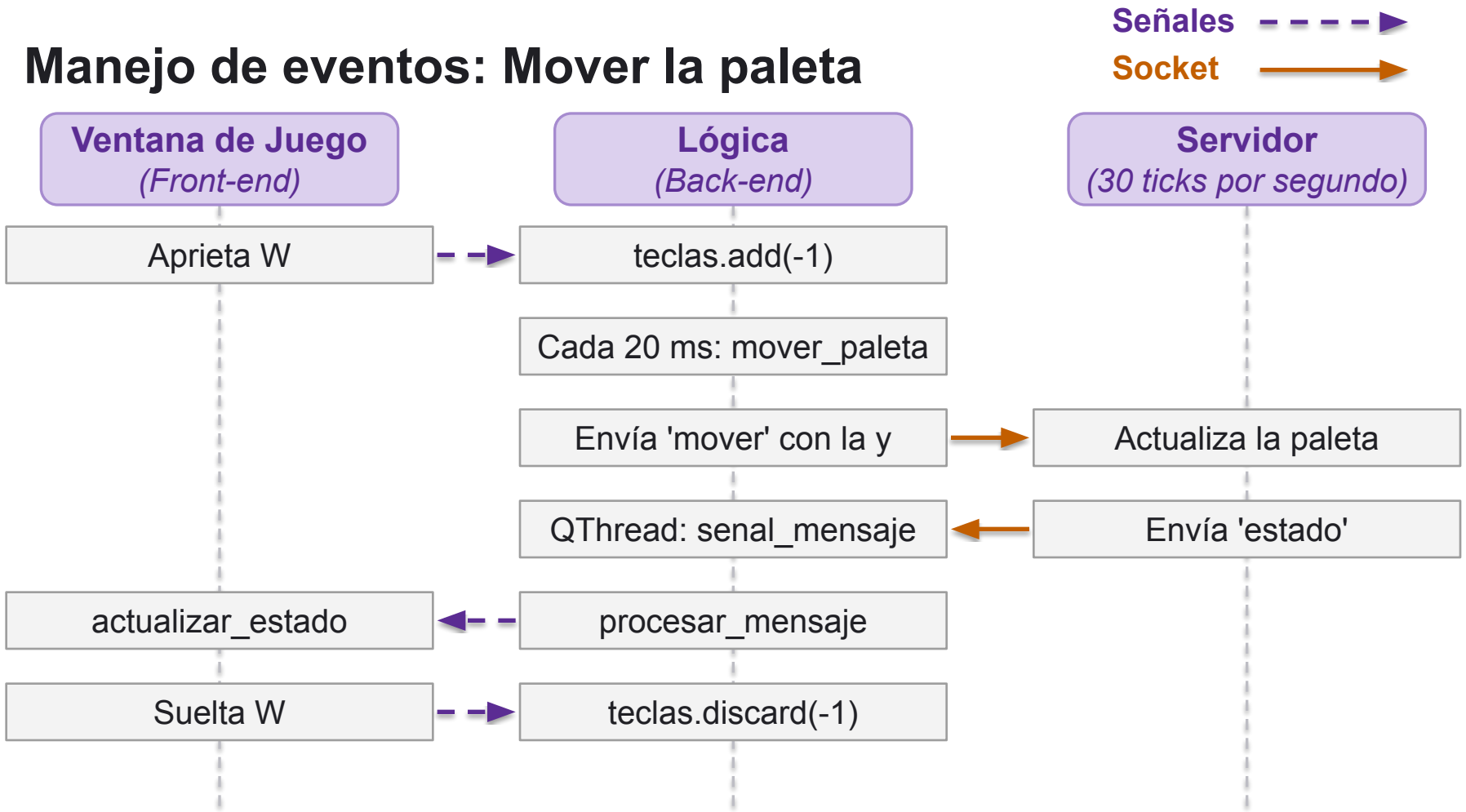
¿Puede el QThread modificar la interfaz?

No. Emite una señal y Qt ejecuta el método conectado **en el thread principal**.

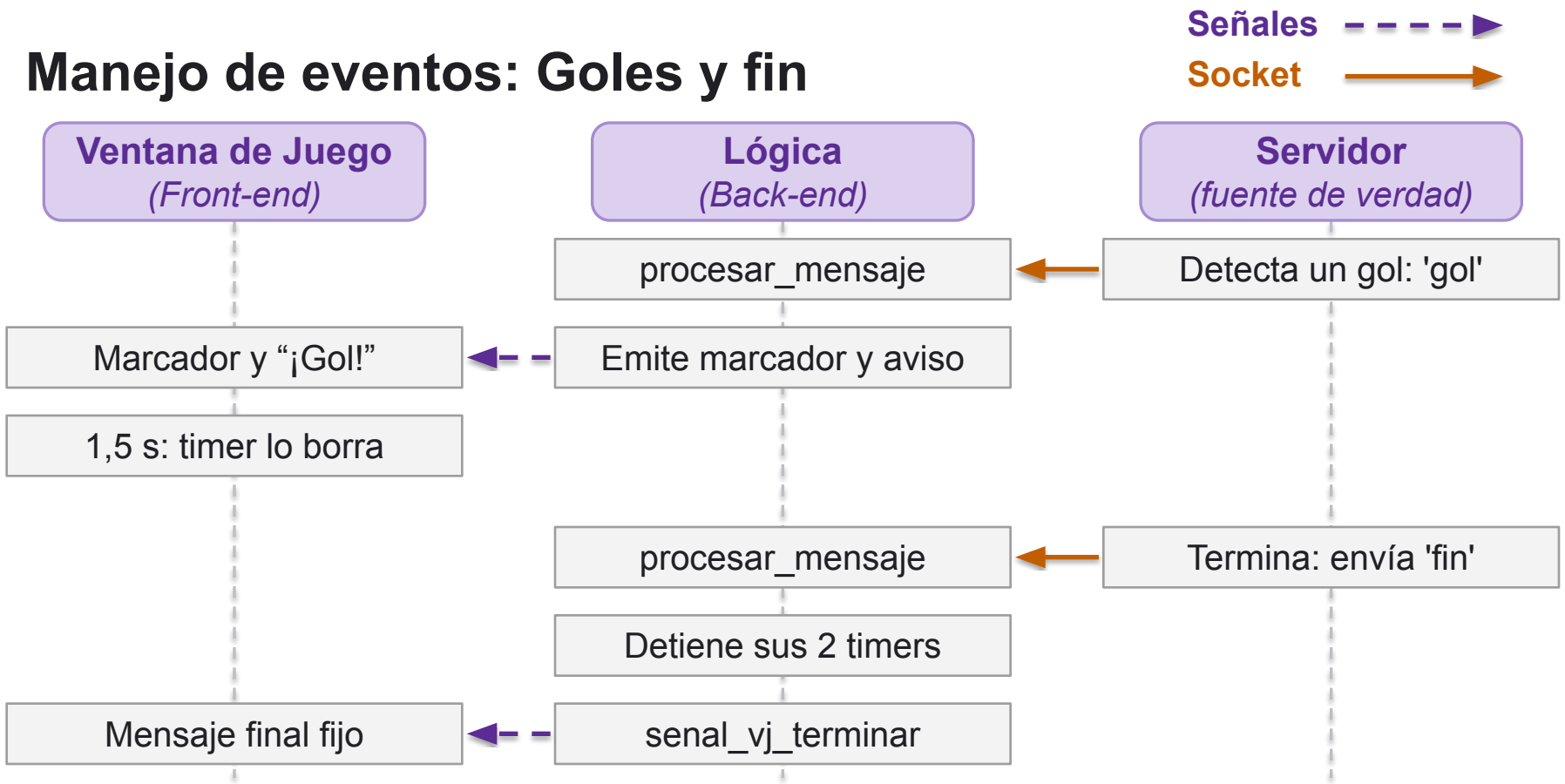
Lógica.conectar_a_servidor

```
self.thread_escuchar = EscucharQThread(self.socket_cliente)
self.thread_escuchar.senal_mensaje.connect(self.procesar_mensaje)
self.thread_escuchar.start()  # start, no run
```

Manejo de eventos: Mover la paleta



Manejo de eventos: Goles y fin



`'rival_desconectado'` sigue el mismo camino que 'fin'.

Los QTimer de la actividad

timer_movimiento · Lógica, cada 20 ms

```
self.timer_movimiento.setInterval(p.TICK_MOVIMIENTO)
self.timer_movimiento.timeout.connect(self.mover_paleta)
```

timer_reloj · Lógica, cada 1 segundo

```
self.timer_reloj.setTimerType(Qt.TimerType.PreciseTimer)
self.timer_reloj.setInterval(1000)
self.timer_reloj.timeout.connect(self.avanzar_reloj)
```

timer_mensaje · VentanaJuego, una sola vez

```
self.timer_mensaje.setSingleShot(True)
self.timer_mensaje.setInterval(p.DURACION_MENSAJE)
self.timer_mensaje.timeout.connect(self.label_mensaje.clear)
```

Se configuran **una sola vez, en el __init__**. Después solo se llama a start() y stop().

Eventos de teclado: *Front-end*

VentanaJuego

```
TECLAS = {Qt.Key.Key_W: -1, Qt.Key.Key_S: 1}

def keyPressEvent(self, evento: QKeyEvent):
    if (evento.key() in self.TECLAS
            and not evento.isAutoRepeat()):
        direccion = self.TECLAS[evento.key()]
        self.senal_tecla.emit(direccion, True)

def keyReleaseEvent(self, evento: QKeyEvent):
    ...  # igual, pero emite False
```

keyPressEvent recibe el evento: de ahí sacamos la tecla.

isAutoRepeat(): al mantener una tecla, el sistema repite el evento.

Eventos de teclado: *Back-end*

Lógica.mover_paleta

```
def mover_paleta(self):
    direccion = sum(self.teclas)  # W + S se anulan
    if direccion == 0:
        return
    limite = p.ALTO_CANCHA - p.ALTO_PALETA
    paso = direccion * p.VELOCIDAD_PALETA
    nuevo_y = max(0, min(self.y + paso, limite))
    if nuevo_y != self.y:
        self.y = nuevo_y
        self.enviar_comando({'comando': 'mover', 'y': self.y})
```

La paleta la mueve el timer, no el evento: así la velocidad es fija y fluida.

Cerrar bien la conexión

¿Qué pasa si cerramos la ventana y nada más?

El QThread sigue bloqueado en recv y se destruye mientras corre: *QThread: Destroyed while thread is still running*

¿Dónde cerramos el socket?

En main.py, conectando **app.aboutToQuit**, que se emite justo antes de que termine la aplicación.

¿Por qué shutdown antes de close?

shutdown despierta al recv del otro thread: recibir lanza una excepción y run termina. Luego **wait()** espera a que el thread acabe.

main.py

```
app.aboutToQuit.connect(logica.cerrar)
```

Cerrar bien la conexión

Lógica.cerrar

```
def cerrar(self):
    self.timer_movimiento.stop()
    self.timer_reloj.stop()
    try:
        self.socket_cliente.shutdown(socket.SHUT_RDWR)
    except OSError:
        pass
    self.socket_cliente.close()
    if self.thread_escuchar is not None:
        self.thread_escuchar.wait()
```

Bonus: Pausa

La tecla P solo **pide** la pausa. El servidor avisa con tipo 'pausa' a **ambos** clientes, y recién ahí se detienen los timers.

Lógica.cambiar_pausa

```
def cambiar_pausa(self, activa, nombre):
    if activa:
        self.timer_movimiento.stop()
        self.timer_reloj.stop()
    else:
        self.timer_movimiento.start()
        self.timer_reloj.start()
    self.senal_vj_pausa.emit(activa, nombre)
```

Bonus: Revancha

La revancha vuelve a pasar por empezar_partida. Los timers **ya están conectados**: solo se reinicia el estado.

Lógica.empezar_partida

```
def empezar_partida(self, mensaje):
    self.en_ventana_juego = True
    self.y = (p.ALTO_CANCHA - p.ALTO_PALETA) // 2
    self.teclas.clear()
    ...
```

Si conectas timeout aquí, en la revancha queda conectado dos veces: ¡la paleta va al doble de velocidad!